Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : Analyse Comportementale Clientèle Retail

Préparé par Ola Chakroun

Résumé du Projet

Ce projet vise à construire un système complet d'analyse comportementale clientèle pour une entreprise de retail. Trois modèles de Machine Learning ont été développés : classification du churn (Random Forest), segmentation client (KMeans), et prévision des dépenses (Régression Linéaire). L'application web Flask permet aux équipes métier d'interagir avec les modèles en temps réel.

Année Universitaire : 2025-2026

Table des matières

1 Introduction et Contexte du Projet **3**
1.1 Objectifs Métier . . . 3
1.2 Données . . . 3
1.3 Architecture du Projet . . . 3

2 Prétraitement des Données (`preprocessing.py`) **3**
2.1 Vue d'ensemble du Pipeline . . . 3
2.2 Étape 1 : Suppression des Features à Variance Nulle . . . 4
2.3 Étape 2 : Correction des Valeurs Aberrantes . . . 4
2.4 Étape 3 : Parsing de la Date d'Inscription . . . 4
2.5 Étape 4 : Feature Engineering sur l'IP . . . 4
2.6 Étape 5 : Création de Features Comportementales . . . 4
2.7 Étape 6 : Encodage des Variables Catégorielles . . . 5

3 Data Leakage : Problème Critique et Correction **5**
3.1 Qu'est-ce que le Data Leakage ? . . . 5
3.2 Symptôme Initial : AUC = 1.00 (Churn) . . . 5
3.3 Identification des Features Fuyantes . . . 5
3.4 Liste Complète des Features Exclues . . . 5
3.5 Résultats Après Correction . . . 6

4 Modèles de Machine Learning **6**
4.1 Modèle 1 : Classification du Churn (Random Forest) . . . 6
4.1.1 Pourquoi Random Forest ? . . . 6
4.1.2 Pipeline Complet . . . 6
4.1.3 Étapes du Pipeline . . . 6
4.1.4 Évaluation . . . 7
4.1.5 Analyse Détaillée des Performances . . . 7
4.1.6 Top 5 Features Importantes . . . 7
4.2 Modèle 2 : Clustering (KMeans) . . . 8
4.2.1 Objectif . . . 8
4.2.2 Problème Initial : Silhouette = 0.99 . . . 8
4.2.3 Solution Mise en Place . . . 9
4.2.4 Recherche du K Optimal . . . 9
4.2.5 Visualisation des Segments . . . 10
4.2.6 Profil des 4 Segments . . . 10
4.2.7 Analyse PCA Approfondie . . . 10
4.2.8 Projections PCA du Churn . . . 11
4.3 Modèle 3 : Régression (LinearRegression) . . . 12
4.3.1 Objectif . . . 12
4.3.2 Pourquoi Régression Linéaire ? . . . 12
4.3.3 Pipeline . . . 13
4.3.4 Évaluation . . . 13

5 Script d'Inférence et Déploiement **13**
5.1 `predict.py` — Inférence . . . 13
5.2 Application Web Flask (`app.py`) . . . 14
5.2.1 Architecture . . . 14
5.2.2 Exemple d'Utilisation API . . . 14

6 Synthèse des Résultats **14**
6.1 Défis Résolus 14
6.2 Applications Métier 15
6.3 Améliorations Possibles 15

7 Conclusion **15**

1 Introduction et Contexte du Projet

1.1 Objectifs Métier

Le projet répond à quatre questions stratégiques essentielles :

1. **Segmentation Clientèle** : Identifier des groupes homogènes pour des stratégies marketing ciblées
2. **Prédiction du Churn** : Anticiper les départs clients pour des actions de rétention proactives
3. **Prévision de Valeur** : Estimer les dépenses futures pour optimiser les investissements
4. **Déploiement Opérationnel** : Fournir une interface web conviviale (Flask)

1.2 Données

Le dataset `retail_customers_COMPLETE_CATEGORICAL.csv` contient :

— **4372** clients

— **83** features après prétraitement

Table 1 – Catégories de Features

Catégorie	Exemples
RFM	Recency, Frequency, Monetary
Démographiques	Âge, genre, pays, région
Transactionnelles	Nombre d'achats, panier moyen
Engagement	Satisfaction, tickets support
Temporelles	Saisons préférées, ancienneté

1.3 Architecture du Projet

```
ML_Project/
 data/
    raw/              # Données brutes
    processed/        # Données prétraitées
 src/
    preprocessing.py  # Nettoyage & feature engineering
    train_model.py    # Entraînement des 3 modèles
    predict.py        # Inférence
    utils.py          # Fonctions utilitaires
 app/
    app.py            # Application Flask
 models/               # Modèles entraînés (.pkl)
 reports/              # Graphiques et rapports
```

2 Prétraitement des Données (`preprocessing.py`)

2.1 Vue d'ensemble du Pipeline

Le prétraitement transforme les données brutes en features exploitables via 6 étapes :

Pipeline de Prétraitement

1. Suppression des features à variance nulle
2. Correction des valeurs aberrantes
3. Parsing de la date d'inscription
4. Feature engineering sur l'IP
5. Création de nouvelles features comportementales
6. Encodage des variables catégorielles

2.2 Étape 1 : Suppression des Features à Variance Nulle

Objectif : Retirer les colonnes à valeur unique (variance = 0).

```python
def drop_useless_features(df: pd.DataFrame) -> pd.DataFrame:
    to_drop = [col for col in df.columns
               if df[col].nunique(dropna=False) == 1]
    if to_drop:
        df.drop(columns=to_drop, inplace=True)
    return df
```

Listing 1 – Fonction drop_useless_features

Exemple : La colonne `NewsletterSubscribed` contenait uniquement la valeur 1 pour tous les clients → supprimée.

2.3 Étape 2 : Correction des Valeurs Aberrantes

Problème : Codes de valeurs manquantes déguisés en valeurs valides.

Table 2 – Valeurs Aberrantes Détectées

Colonne	Codes	Action
SupportTicketsCount	-1, 999	Remplacés par NaN
SatisfactionScore	-1, 99	Remplacés par NaN

2.4 Étape 3 : Parsing de la Date d'Inscription

Transformation : `RegistrationDate` (texte) → 4 features numériques :

- `RegYear` : Année d'inscription
- `RegMonth` : Mois (1-12)
- `RegDay` : Jour du mois
- `RegWeekday` : Jour de la semaine (0=Lundi)

2.5 Étape 4 : Feature Engineering sur l'IP

Extraction de `LastLoginIP` :

- `IP_IsPrivate` : 1 si IP privée, 0 sinon
- `IP_FirstOctet` : Premier octet (classe réseau)

2.6 Étape 5 : Création de Features Comportementales

Feature clé créée :

$$\texttt{AvgBasketValue} = \frac{\texttt{MonetaryTotal}}{\texttt{Frequency}} \tag{1}$$

Note importante : `MonetaryPerDay` et `TenureRatio` ont été supprimées car dérivées de `Recency` et `CustomerTenureDays` (features fuyantes pour le churn).

2.7 Étape 6 : Encodage des Variables Catégorielles

Table 3 – Stratégies d'Encodage

Type	Variables	Raison
Ordinal	AgeCategory, LoyaltyLevel	Hiérarchie naturelle
One-Hot	RFMSegment, CustomerType	Pas d'ordre
Target Encoding	Country (40 valeurs)	Haute cardinalité

3 Data Leakage : Problème Critique et Correction

3.1 Qu'est-ce que le Data Leakage ?

Le data leakage se produit quand des informations du futur ou de la cible sont accidentellement incluses dans les features d'entraînement. Cela permet au modèle d'obtenir des performances artificiellement parfaites en entraînement, mais il échoue complètement en production.

3.2 Symptôme Initial : AUC = 1.00 (Churn)

Alerte Rouge !

Un AUC de 1.00 signifie que le modèle a séparé parfaitement tous les clients qui ont churné de ceux qui sont restés. C'est impossible sur des données réelles comportementales → le modèle a triché en utilisant des features qui "donnent la réponse".

3.3 Identification des Features Fuyantes

Une analyse de corrélation a révélé 14 features hautement corrélées avec Churn :

Table 4 – Features fuyantes identifiées (Churn)

Feature	Corrélation	Raison
ChurnRiskCategory	0.88	Construite pour prédire le churn
Recency	0.86	Client parti = n'achète plus = Recency élevée
CustomerType_Perdu	0.70	"Perdu" = churné par définition
RFMSegment_Dormants	0.58	"Dormant" = parti
LoyaltyLevel	-0.43	Calculé sur historique récent
CustomerTenureDays	-0.45	Durée courte = client parti
AccountStatus_Closed	—	Compte fermé = parti

3.4 Liste Complète des Features Exclues

```
LEAKY_CHURN_FEATURES = [
    # Proxies temporels directs
    'Recency', 'CustomerTenureDays', 'FirstPurchaseDaysAgo',
    'TenureRatio', 'MonetaryPerDay',
    # Cat gories synth tiques
    'ChurnRiskCategory', 'LoyaltyLevel', 'SpendingCategory',
    # One-Hot de CustomerType
    'CustomerType_Perdu', 'CustomerType_Hyperactif',
    'CustomerType_Nouveau', 'CustomerType_Occasionnel',
    'CustomerType_Regulier',
    # One-Hot de RFMSegment
    'RFMSegment_Champions', 'RFMSegment_Dormants',
```

```
    'RFMSegment_Fideles', 'RFMSegment_Potentiels',
    # One-Hot de AccountStatus
    'AccountStatus_Closed', 'AccountStatus_Suspended',
    'AccountStatus_Active', 'AccountStatus_Pending'
]
```

Listing 2 – LEAKY_CHURN_FEATURES

3.5 Résultats Après Correction

Performances Réalistes

- **AUC Score : 0.958**
- **Accuracy : 88%**
- **Precision (Churn=1) : 0.81**
- **Recall (Churn=1) : 0.85**
- **F1-Score (macro) : 0.87**

Ces métriques sont réalistes et exploitables en production. Le modèle généralise correctement.

4 Modèles de Machine Learning

4.1 Modèle 1 : Classification du Churn (Random Forest)

4.1.1 Pourquoi Random Forest ?

- Robuste aux outliers (arbres de décision)
- Gère les non-linéarités sans feature engineering manuel
- Feature importance intégrée
- Performant avec peu de tuning

4.1.2 Pipeline Complet

```
churn_pipeline = ImbPipeline([
    ('imputer', KNNImputer(n_neighbors=5)),
    ('scaler', RobustScaler()),
    ('smote', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(
        n_estimators=200,
        max_depth=10,
        min_samples_leaf=10,
        max_features='sqrt',
        class_weight='balanced',
        random_state=42
    )),
])
```

Listing 3 – Pipeline Churn

4.1.3 Étapes du Pipeline

KNNImputer Remplit les NaN par la moyenne des 5 voisins les plus proches

RobustScaler Normalise avec médiane et IQR (robuste aux outliers)

SMOTE Crée des exemples synthétiques de la classe minoritaire (Churn=1)

Random Forest 200 arbres, profondeur max 10, `class_weight='balanced'`

4.1.4 Évaluation

Table 5 – Résultats Classification Churn

	Precision	**Recall**	**F1-Score**
Classe 0 (Fidèle)	0.92	0.90	0.91
Classe 1 (Churn)	0.81	0.85	0.83
Macro Avg	0.86	0.87	0.87

AUC : 0.958

La performance du modèle est illustrée par la matrice de confusion et la courbe ROC (Figure 1).

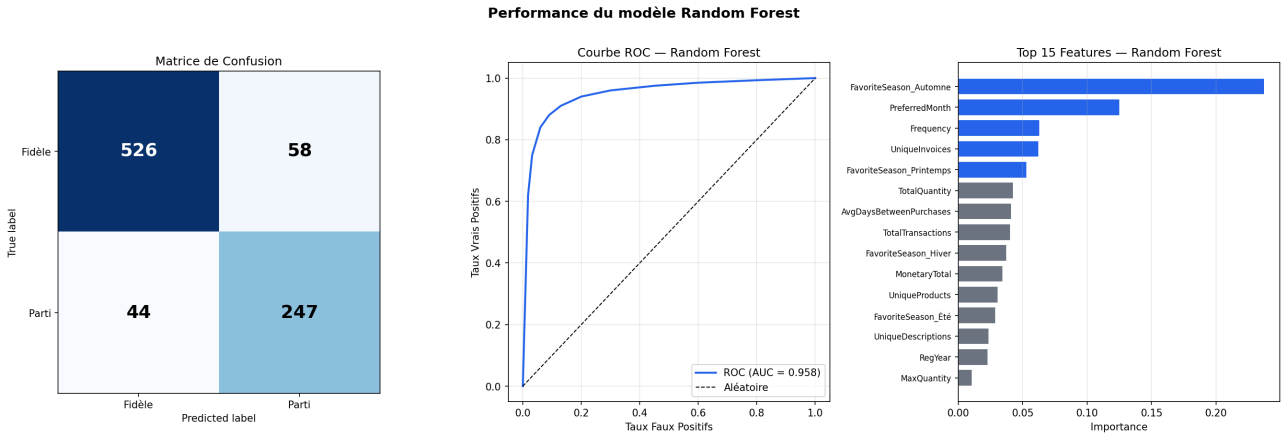

Figure 1 – Performance du modèle Random Forest - Matrice de Confusion et Courbe ROC

4.1.5 Analyse Détaillée des Performances

La Figure 2 présente une analyse approfondie des performances du classifieur avec la courbe ROC et la courbe Précision-Rappel.

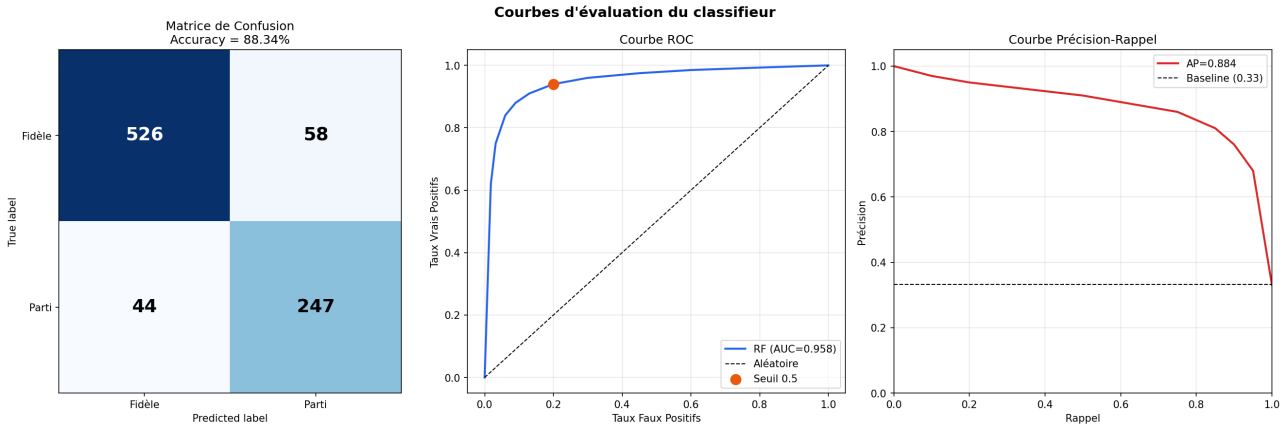

Figure 2 – Courbes d'évaluation du classifieur - ROC et Précision-Rappel

4.1.6 Top 5 Features Importantes

1. `FavoriteSeason_Automne` — 0.2377
2. `PreferredMonth` — 0.1255
3. `Frequency` — 0.0634

4. `UniqueInvoices` — 0.0624
5. `FavoriteSeason_Printemps` — 0.0531

L'importance complète des variables est présentée en Figure 3.

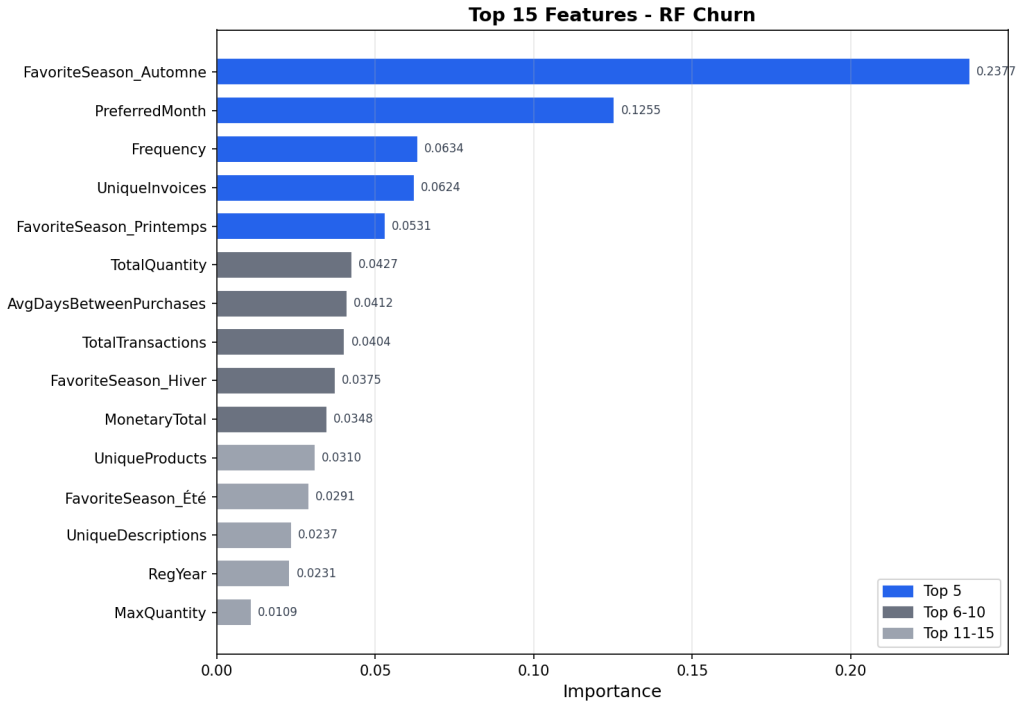

FIGURE 3 – Top 15 Feature Importances — Random Forest

4.2 Modèle 2 : Clustering (KMeans)

4.2.1 Objectif

Segmenter les clients en 4 groupes homogènes sans supervision :

1. VIP : Meilleurs clients
2. Fidèle : Bons clients réguliers
3. Occasionnel : Clients irréguliers
4. À Risque : Clients inactifs ou à faible valeur

4.2.2 Problème Initial : Silhouette = 0.99

Silhouette de 0.99 = clusters triviaux. KMeans séparait "clients avec outliers" vs "clients normaux" — inutilisable métier.

Cause racine : PCA révèle que PC1 explique 97.6% de la variance, dominée par `MinQuantity` (-80995 à +80995).

La variance expliquée par les composantes PCA est illustrée en Figure 4.

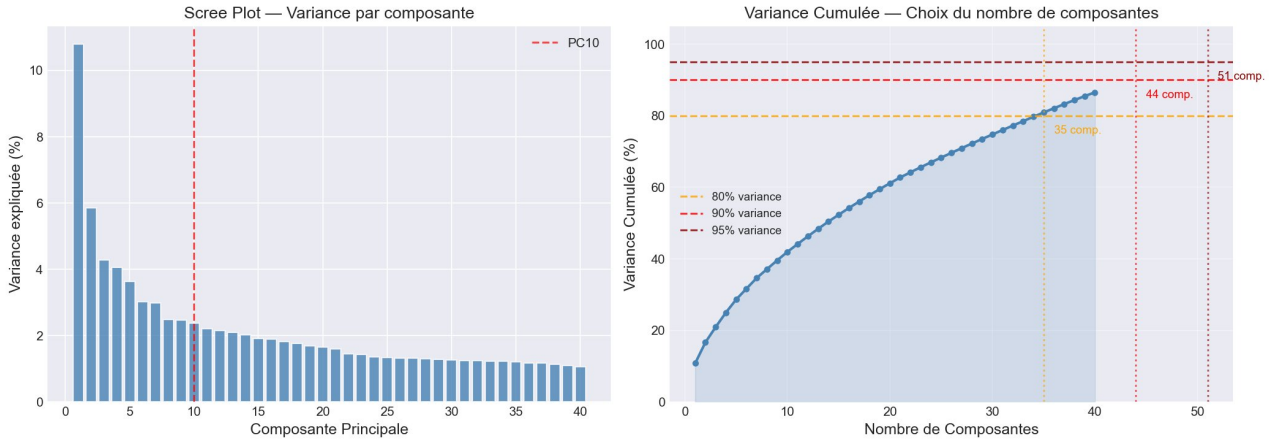

FIGURE 4 – Variance expliquée par les composantes PCA - Scree Plot et Variance Cumulée

4.2.3 Solution Mise en Place

1. **Sélection** : 9 features RFM uniquement
2. **Filtrage** : Exclure 52 clients avec `MonetaryTotal` ≤ 0
3. **Winsorisation** : Clip aux percentiles 1 et 99
4. **Log-transform** : `np.log1p()` sur colonnes asymétriques
5. **RobustScaler** : Normalisation finale

4.2.4 Recherche du K Optimal

La méthode du coude et le score de silhouette ont permis de valider le choix de k=4 (Figure 5).

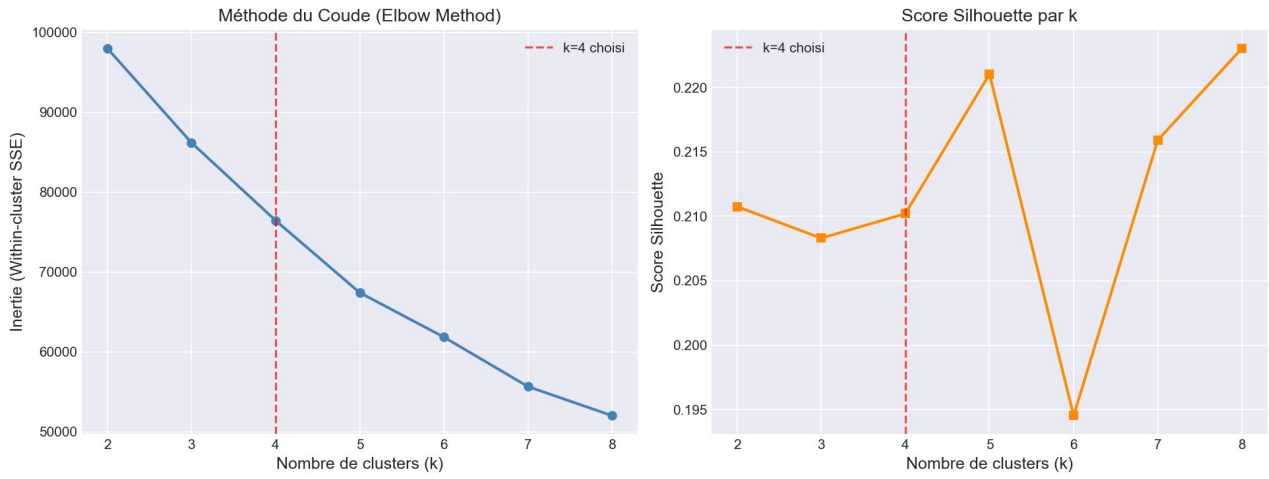

FIGURE 5 – Méthode du Coude et Score de Silhouette pour la sélection de k

TABLE 6 – Méthode du Coude + Silhouette

K	Silhouette	Inertie	Commentaire
2	0.622	49 103 368	Trop élevé — trivial
3	0.406	32 640 758	Encore élevé
4	**0.228**	**20 586 522**	**Coude + interprétable**
5	0.215	12 180 263	Gain marginal

4.2.5 Visualisation des Segments

La visualisation des clusters en 2D après PCA est présentée en Figure 6.

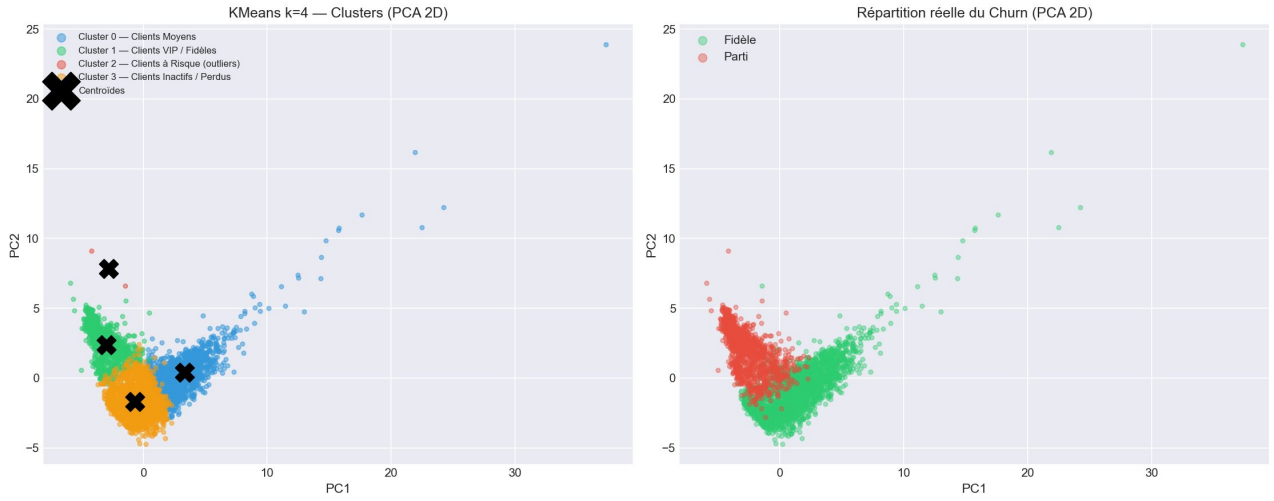

FIGURE 6 – Visualisation des 4 segments clients en projection PCA 2D

4.2.6 Profil des 4 Segments

L'analyse des profils de segments est présentée en Figure 7.

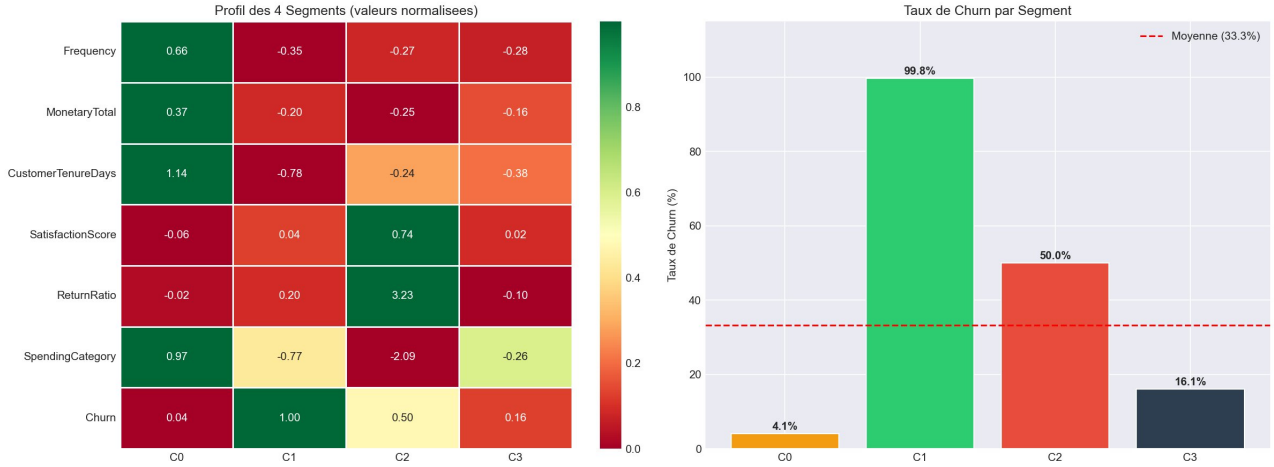

FIGURE 7 – Profil des 4 segments - Caractéristiques normalisées et taux de churn

TABLE 7 – Profil des Clusters (k=4)

Segment	Monetary	Recency	Frequency	Churn	n
VIP	3 660€	34j	8.8	8.3%	1 623
Fidèle	2 708€	79j	8.1	27.8%	461
Occasionnel	1 795€	109j	5.9	42.6%	136
À Risque	420€	135j	1.6	52.1%	2 100

Validation métier : Gradient cohérent — churn croît de VIP → À Risque.

4.2.7 Analyse PCA Approfondie

Les contributions des variables aux composantes principales sont illustrées dans les Figures 8 et 9.

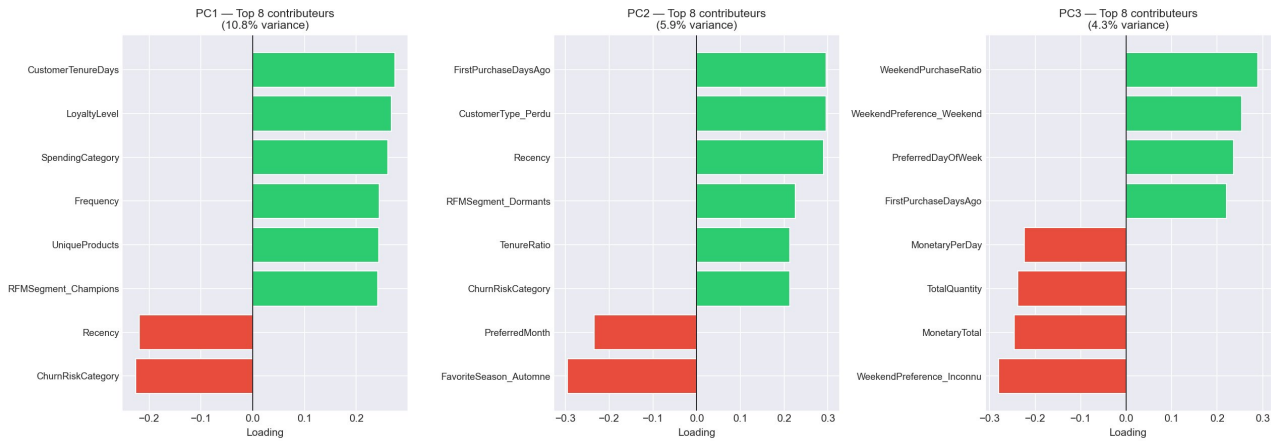

FIGURE 8 – Contributions des variables aux 3 premières composantes principales

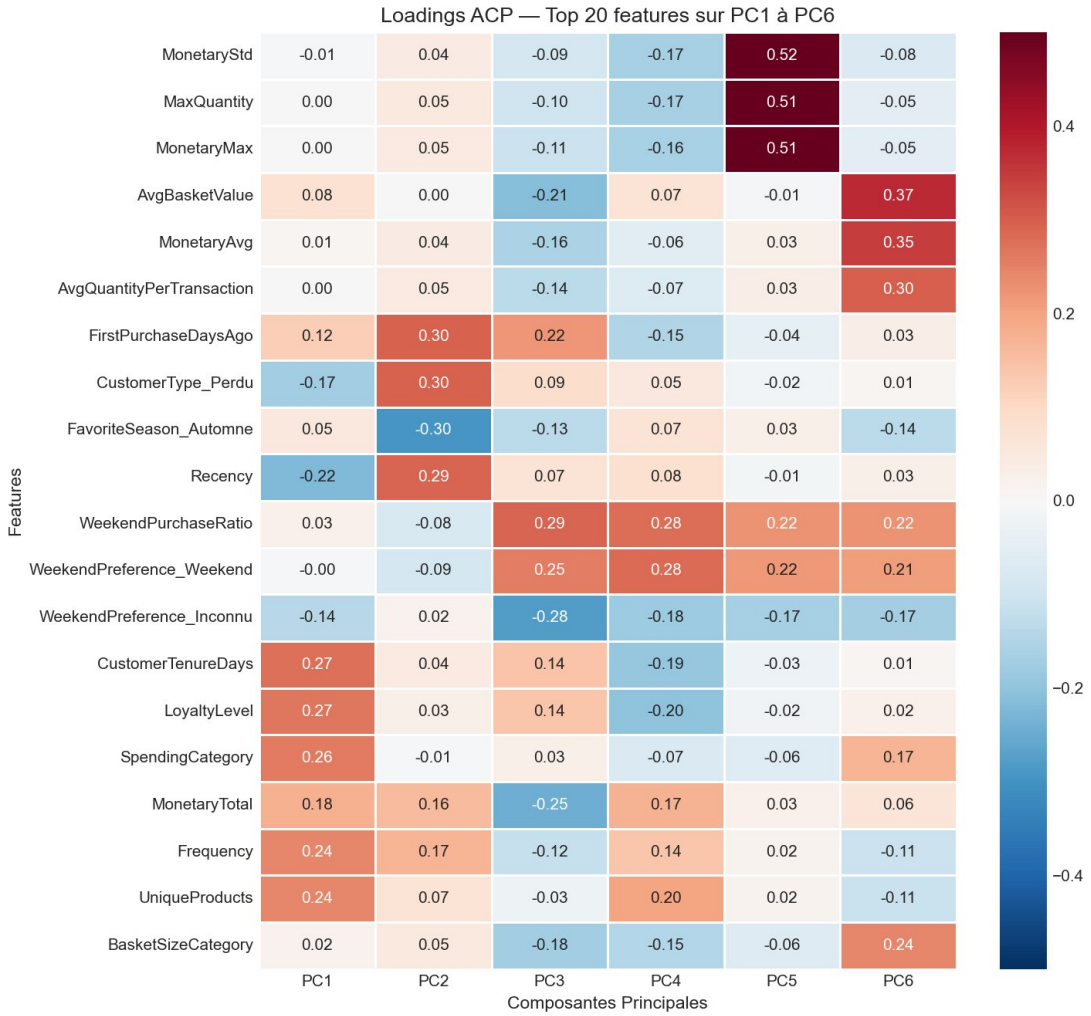

FIGURE 9 – Heatmap des loadings PCA - Top 20 features sur PC1 à PC6

4.2.8 Projections PCA du Churn

Les Figures 10 et 11 montrent la distribution des clients churned vs fidèles dans l'espace PCA.

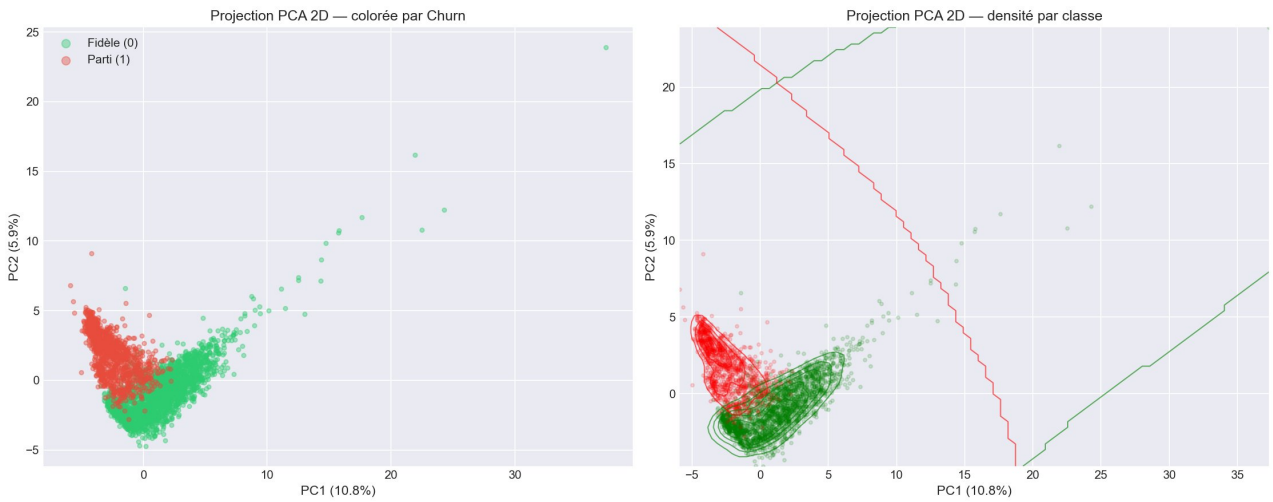

FIGURE 10 – Projection PCA 2D des clients colorée par statut Churn avec densités

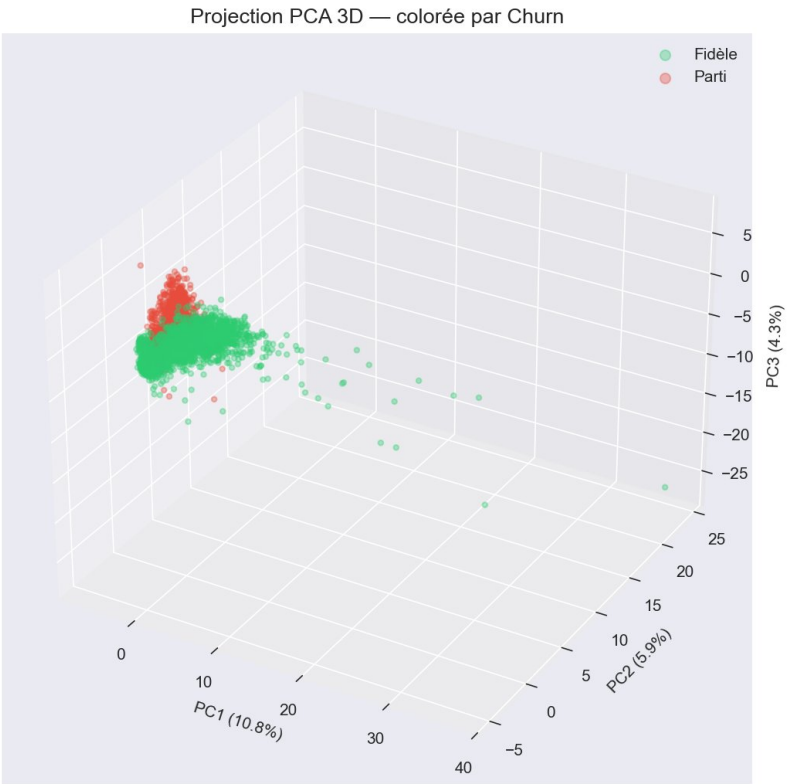

FIGURE 11 – Projection PCA 3D des clients colorée par statut Churn

4.3 Modèle 3 : Régression (LinearRegression)

4.3.1 Objectif

Prédire le montant total des dépenses futures (`MonetaryTotal`).

4.3.2 Pourquoi Régression Linéaire ?

- Baseline simple et interprétable
- Coefficients = impact direct de chaque feature
- Entraînement très rapide

— Pas de risque d'overfitting

4.3.3 Pipeline

```
regression_pipeline = Pipeline([
    ('imputer', KNNImputer(n_neighbors=5)),
    ('scaler', RobustScaler()),
    ('regressor', LinearRegression()),
])
```

Listing 4 – Pipeline Régression

Pas de SMOTE : Régression = prédiction de montant continu, pas classification.

4.3.4 Évaluation

— **R^2 = 0.668** : Le modèle explique 66.8% de la variance
— **MAE = 863.83 GBP** : Erreur moyenne de ±864£

L'évaluation visuelle du modèle (Réel vs Prédit) est présentée en Figure 12.

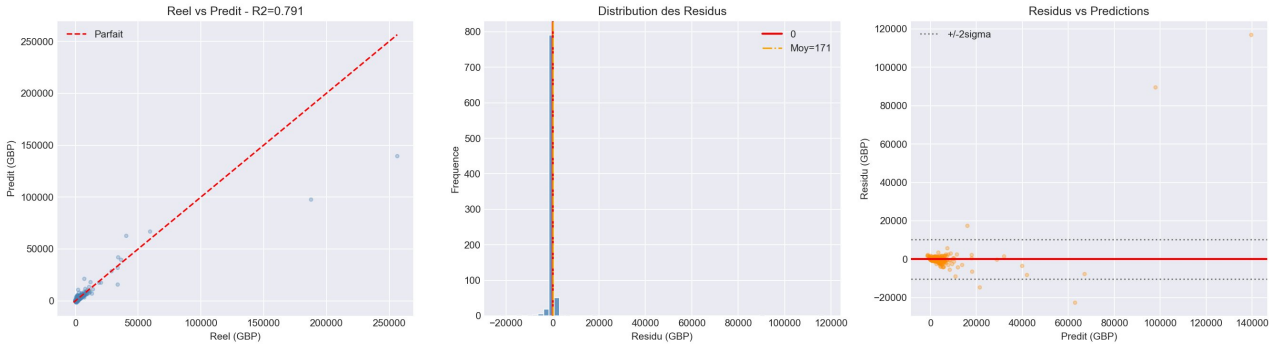

FIGURE 12 – Évaluation de la régression - Prédictions vs Valeurs réelles avec analyse des résidus

Une analyse complémentaire des performances de régression est présentée en Figure 13.

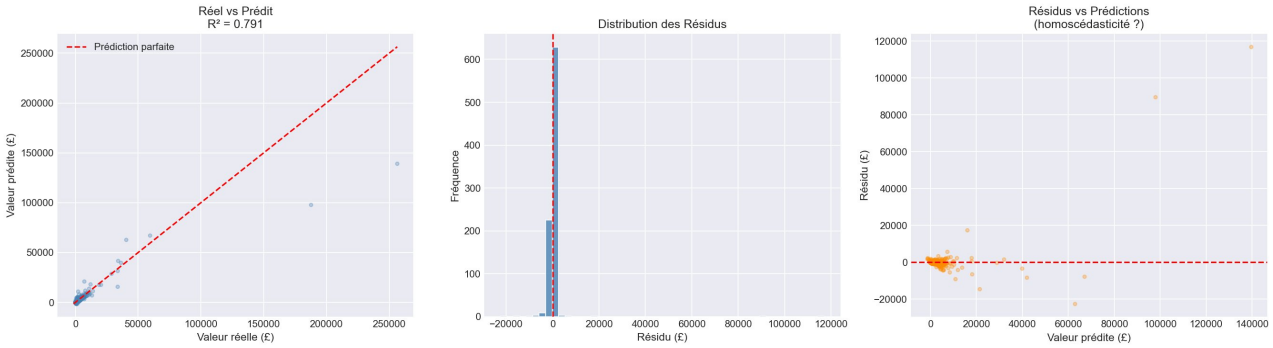

FIGURE 13 – Analyse détaillée de la régression - Distribution et analyse des résidus

5 Script d'Inférence et Déploiement

5.1 `predict.py` — Inférence

Le script permet de faire des prédictions sur de nouvelles données en utilisant les modèles entraînés.

```
churn_pipeline = joblib.load('churn_pipeline.pkl')
clustering_artifacts = joblib.load('clustering_preprocessor_pipeline.pkl')
kmeans_model = joblib.load('kmeans_model.pkl')
regression_pipeline = joblib.load('regression_pipeline.pkl')
```

Listing 5 – Chargement des modèles

5.2 Application Web Flask (app.py)

5.2.1 Architecture

L'application fournit :

1. **Formulaire de prédiction** (/predict) : Saisie des features → 3 prédictions
2. **Dashboard de métriques** (/) : Performances des modèles
3. **Visualisation des segments** (/segments) : Profils détaillés
4. **API JSON** (/api/predict) : Intégration CRM/ERP

5.2.2 Exemple d'Utilisation API

```
curl -X POST http://localhost:5000/api/predict \
  -H "Content-Type: application/json" \
  -d '{"Frequency":5,"MonetaryTotal":1200,"Age":35,...}'
```

Listing 6 – Requête cURL

```
{
  "churn": 0,
  "probability": 0.15,
  "risk": "Faible",
  "cluster": 1,
  "cluster_name": "Fidele",
  "monetary_prediction": 1850.25
}
```

Listing 7 – Réponse JSON

6 Synthèse des Résultats

Table 8 – Performances Finales des Modèles

Modèle	Métrique	Valeur	Interprétation
3*Random Forest (Churn)	AUC	0.958	Excellent
	F1-Score	0.87	Très bon
	Recall	0.85	Détecte 85% des churners
3*KMeans (Clustering)	Silhouette	0.228	Acceptable
	k optimal	4	4 segments métier
	VIP Churn	8.3%	Cohérent
2*LinearRegression	R^2	0.668	Acceptable
	MAE	864 GBP	Erreur moyenne

6.1 Défis Résolus

1. **Data Leakage Churn** : 14 features exclues → AUC de 1.00 à 0.958
2. **Clustering Trivial** : Winsorisation + log-transform → Silhouette réaliste

3. **Déséquilibre de Classes** : SMOTE + `class_weight='balanced'`
4. **Imputation Sans Fuite** : KNNImputer dans les pipelines

6.2 Applications Métier

— **Segmentation** : Identifier les 1623 VIP pour offres premium
— **Rétention** : Cibler les 2100 clients À Risque (52% churn)
— **Prévision Budgétaire** : Estimer revenus futurs (MAE = 864£)
— **Automatisation** : API JSON pour intégration CRM/ERP

6.3 Améliorations Possibles

1. Churn : Tester XGBoost/LightGBM (AUC potentiel > 0.97)
2. Clustering : Ajouter features temporelles (saisonnalité)
3. Régression : Passer à Random Forest Regressor (R^2 potentiel 0.75+)
4. Déploiement : Conteneuriser avec Docker
5. Monitoring : Système de détection de drift en production

7 Conclusion

Ce projet a permis de construire un système complet d'analyse comportementale clientèle avec trois modèles ML opérationnels. La rigueur dans la gestion du data leakage et l'adaptation des pipelines spécifiques par modèle garantissent des performances réalistes et exploitables en production.

L'application web Flask transforme ces modèles en outils décisionnels accessibles aux équipes métier, permettant des actions marketing ciblées et proactives basées sur les données.

Points Clés du Projet

— **3 modèles ML** performants et complémentaires
— **88% de précision** sur la prédiction du churn
— **4 segments clients** actionnables identifiés
— **R^2 de 0.67** pour la prévision des dépenses
— **Application Flask** déployable en production
— **Gestion rigoureuse** du data leakage et des features fuyantes